

Guía de Desarrollo de un Videojuego en Greenfoot

Propósito de la guía

Esta actividad busca que el estudiante construya un personaje jugable en Greenfoot y, al mismo tiempo, comprenda que las decisiones de implementación responden a **drivers arquitectónicos**, **concerns** y **trade-offs**. El personaje deberá caminar mediante una secuencia de sprites y saltar siguiendo una trayectoria parabólica modelada mediante una ecuación cuadrática.

1. Objetivos de aprendizaje

Al finalizar el laboratorio, el estudiante será capaz de:

- identificar drivers arquitectónicos básicos en un videojuego 2D;
- reconocer concerns de jugabilidad, mantenibilidad, respuesta visual y simplicidad;
- analizar tensiones entre distintos concerns;
- representar el comportamiento del jugador mediante estados;
- asociar estados del jugador a diferentes imágenes o sprites;
- implementar una animación de caminata mediante un arreglo de imágenes;
- modelar el salto usando una ecuación cuadrática;
- integrar movimiento horizontal, animación y movimiento vertical;
- extender el prototipo incorporando nuevos estados y plataformas.

2. Antes de programar: drivers arquitectónicos

Antes de escribir código conviene identificar qué aspectos del sistema **condicionan las principales decisiones de diseño**. En arquitectura de software, estos elementos suelen denominarse **architectural drivers**.

Un driver arquitectónico es una necesidad, restricción o atributo de calidad que influye de forma importante en la estructura de la solución.

En este videojuego, los drivers principales son los siguientes.

Driver	Necesidad	Consecuencia en el diseño
Jugabilidad	El jugador debe responder de forma inmediata a las teclas y transmitir una sensación clara de control.	Separar la lectura del teclado, el movimiento y la actualización visual.
Respuesta visual	El personaje debe mostrar visualmente si está quieto, caminando o saltando.	Utilizar estados y asociar cada estado a uno o más sprites.
Movimiento natural	El salto no debe ser simplemente mover al personaje hacia arriba y luego hacia abajo de forma arbitraria.	Modelar la posición vertical mediante una función dependiente del tiempo.
Mantenibilidad	El código debe poder extenderse con nuevos estados, animaciones o comportamientos.	Dividir la lógica en métodos con responsabilidades claras y utilizar un <code>enum</code> para representar estados.
Simplicidad	El proyecto debe ser comprensible para estudiantes que están comenzando con programación orientada a objetos.	Evitar motores físicos complejos y utilizar una ecuación cuadrática explícita.
Extensibilidad	Posteriormente se deben poder incorporar caída, plataformas, enemigos o ataques.	Diseñar la clase <code>Jugador</code> de forma modular y desacoplada.
Rendimiento	La animación debe ser fluida sin cambiar de sprite a una velocidad excesiva.	Controlar la frecuencia de cambio de frames mediante un contador.

3. Concerns del sistema

Los **concerns** son intereses o aspectos que deben ser atendidos durante el diseño del sistema. En este caso, algunos concerns relevantes son:

- **Control:** el jugador debe responder a las entradas del teclado.
- **Representación visual:** la imagen debe reflejar lo que el personaje está haciendo.
- **Realismo:** el salto debe parecer continuo y natural.
- **Comprensibilidad:** el código debe ser fácil de leer y modificar.
- **Mantenibilidad:** agregar nuevos estados no debería requerir reescribir toda la clase.
- **Fluidez:** la animación debe cambiar de sprite a una velocidad adecuada.
- **Consistencia:** el estado lógico del jugador debe coincidir con lo que se muestra en pantalla.

4. Tensiones entre concerns

En arquitectura de software no siempre es posible maximizar todos los concerns al mismo tiempo. Frecuentemente existen **tensiones** o **trade-offs**.

4.1. Realismo versus simplicidad

Un motor físico completo podría considerar velocidad, aceleración, fricción, colisiones y fuerzas. Sin embargo, para esta actividad se utiliza:

$$y(t) = y_0 - v_0 t + \frac{1}{2} g t^2$$

Esto sacrifica parte del realismo físico a cambio de una solución:

- más simple;
- explicable matemáticamente;
- suficiente para un juego 2D básico;
- apropiada para un laboratorio introductorio.

4.2. Fluidez versus cantidad de actualizaciones

Cambiar el sprite en cada ejecución de `act()` puede provocar una animación demasiado rápida. Por ello se introduce un contador:

$$\text{act()} \rightarrow \text{contador} \rightarrow \text{cambio de frame}$$

La animación no se actualiza en cada ciclo, sino después de un número determinado de ciclos.

4.3. Simplicidad versus extensibilidad

Podríamos implementar el comportamiento utilizando solamente varios `if`. Sin embargo, a medida que aparecen nuevos comportamientos:

QUIETO, CAMINANDO, SALTANDO, CAYENDO, ATACANDO

el código se vuelve más difícil de mantener.

Por ello se utiliza un estado explícito mediante un `enum`.

4.4. Control versus consistencia visual

El personaje podría moverse correctamente, pero si su imagen permanece siempre igual, la experiencia visual sería inconsistente.

Por tanto, se separan dos responsabilidades:

Movimiento espacial + Animación visual

y ambas se coordinan mediante el estado del jugador.

5. Decisiones de diseño resultantes

A partir de los drivers y concerns anteriores se adoptan las siguientes decisiones:

1. representar el comportamiento del jugador mediante estados;
2. asociar los estados a sprites;
3. almacenar secuencias de sprites en arreglos;
4. utilizar un contador para controlar la velocidad de animación;
5. separar movimiento horizontal y movimiento vertical;
6. modelar el salto mediante una ecuación cuadrática;
7. utilizar métodos pequeños con responsabilidades específicas.

La relación general será:

Entrada → Estado → Comportamiento → Representación visual

6. Parte I: creación del escenario

6.1. Paso 1: crear un nuevo escenario Greenfoot

Crear un nuevo escenario y una clase de mundo llamada Mundo.

```
1 import greenfoot.*;
2
3 public class Mundo extends World
4 {
5     public Mundo()
6     {
7         super(800, 500, 1);
8         prepararMundo();
9     }
10
11     private void prepararMundo()
12     {
13         Jugador jugador = new Jugador();
14         addObject(jugador, 100, 400);
15     }
16 }
```

El mundo tendrá:

- 800 píxeles de ancho;
- 500 píxeles de alto;
- un jugador inicialmente ubicado en (100,400).

7. Parte II: estados y sprites

7.1. Paso 2: definir los estados del jugador

Un personaje puede encontrarse en diferentes estados:

- QUIETO;
- CAMINANDO_DERECHA;
- CAMINANDO_IZQUIERDA;
- SALTANDO.

En Java podemos representarlos mediante:

```
1 private enum Estado
2 {
3     QUIETO ,
4     CAMINANDO_DERECHA ,
5     CAMINANDO_IZQUIERDA ,
6     SALTANDO
7 }
```

Luego declaramos:

```
1 private Estado estado = Estado.QUIETO;
```

Esta decisión evita utilizar valores ambiguos como:

```
1 estado = 1;  
2 estado = 2;  
3 estado = 3;
```

7.2. Paso 3: preparar los sprites

Agregar a la carpeta `images` del proyecto:

idle.png
walk1.png
walk2.png
walk3.png
walk4.png

La secuencia de caminata será:

$$Sprite_1 \rightarrow Sprite_2 \rightarrow Sprite_3 \rightarrow Sprite_4 \rightarrow Sprite_1$$

Declaramos:

```
1 private GreenfootImage imagenQuieto;  
2 private GreenfootImage[] caminar;
```

7.3. Paso 4: cargar las imágenes

```
1 private void cargarImagenes()  
2 {  
3     imagenQuieto = new GreenfootImage("idle.png");  
4  
5     caminar = new GreenfootImage[4];  
6  
7     caminar[0] = new GreenfootImage("walk1.png");  
8     caminar[1] = new GreenfootImage("walk2.png");  
9     caminar[2] = new GreenfootImage("walk3.png");  
10    caminar[3] = new GreenfootImage("walk4.png");  
11  
12    setImage(imagenQuieto);  
13 }
```

En el constructor:

```
1 public Jugador()  
2 {  
3     cargarImagenes();  
4 }
```

8. Parte III: movimiento horizontal

8.1. Paso 5: detectar las teclas

Greenfoot permite consultar una tecla mediante:

```
1 Greenfoot.isKeyDown("right")
```

Creamos:

```
1 private void controlarMovimientoHorizontal()
2 {
3     if (Greenfoot.isKeyDown("right"))
4     {
5         setLocation(getX() + 4, getY());
6
7         if (!saltando)
8         {
9             estado = Estado.CAMINANDO_DERECHA;
10        }
11    }
12    else if (Greenfoot.isKeyDown("left"))
13    {
14        setLocation(getX() - 4, getY());
15
16        if (!saltando)
17        {
18            estado = Estado.CAMINANDO_IZQUIERDA;
19        }
20    }
21    else
22    {
23        if (!saltando)
24        {
25            estado = Estado.QUIETO;
26        }
27    }
28 }
```

La relación es:

Flecha derecha $\rightarrow$ *CAMINANDO_DERECHA* $\rightarrow x = x + 4$

8.2. Paso 6: entender movimiento lógico y movimiento visual

El movimiento espacial se obtiene mediante:

```
1 setLocation(getX() + 4, getY());
```

Mientras que el movimiento visual se produce al cambiar:

walk1.png
walk2.png
walk3.png
walk4.png

Por tanto:

$$\text{Movimiento del videojuego} = \text{Movimiento espacial} + \text{Animación visual}$$

9. Parte IV: animación mediante sprites

9.1. Paso 7: controlar el frame actual

Declaramos:

```
1 private int frameActual = 0;
2 private int contadorAnimacion = 0;
```

Luego implementamos:

```
1 private void animarCaminata(boolean izquierda)
2 {
3     contadorAnimacion++;
4
5     if (contadorAnimacion >= 6)
6     {
7         frameActual++;
8
9         if (frameActual >= caminar.length)
10        {
11            frameActual = 0;
12        }
13
14        GreenfootImage imagen =
15            new GreenfootImage(caminar[frameActual]);
16
17        if (izquierda)
18        {
19            imagen.mirrorHorizontally();
20        }
21
22        setImage(imagen);
23        contadorAnimacion = 0;
24    }
25 }
```

El valor 6 controla la frecuencia de actualización.

Valor	Efecto
3	Animación rápida
6	Animación intermedia
10	Animación lenta

9.2. Paso 8: asociar estados a imágenes

```
1 private void actualizarAnimacion()
2 {
3     if (estado == Estado.QUIETO)
4     {
5         setImage(imagenQuieto);
```



```

6      }
7      else if (estado == Estado.CAMINANDO_DERECHA)
8      {
9          animarCaminata(false);
10     }
11     else if (estado == Estado.CAMINANDO_IZQUIERDA)
12     {
13         animarCaminata(true);
14     }
15 }

```

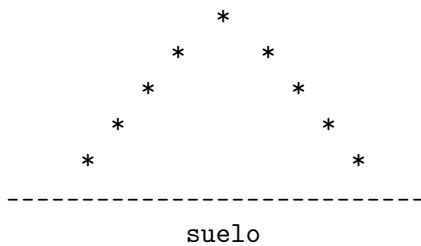
Ahora existe una asociación explícita:

Estado $\rightarrow$ Sprite

10. Parte V: modelar el salto

10.1. Paso 9: comprender la parábola

Queremos que el personaje siga una trayectoria aproximada como:



Esta trayectoria puede representarse con:

$$y(t) = y_0 - v_0 t + \frac{1}{2} g t^2$$

donde:

- y_0 : posición vertical inicial;
- v_0 : velocidad inicial;
- g : gravedad;
- t : tiempo transcurrido.

Es una ecuación cuadrática porque contiene:

$$t^2$$

y corresponde a la forma:

$$y = at^2 + bt + c$$

con:

$$a = \frac{g}{2}, \quad b = -v_0, \quad c = y_0.$$

10.2. Paso 10: considerar las coordenadas de Greenfoot

En Greenfoot, el origen se encuentra arriba a la izquierda.

Por tanto:

- aumentar x mueve hacia la derecha;
- aumentar y mueve hacia abajo;
- disminuir y mueve hacia arriba.

Por esa razón el término de velocidad inicial aparece como:

$$-v_0 t$$

10.3. Paso 11: declarar las variables del salto

```

1 private boolean saltando = false;
2
3 private int tiempoSalto = 0;
4 private int yInicial;
5
6 private double velocidadInicial = 12.0;
7 private double gravedad = 1.0;
```

Interpretación:

- velocidadInicial: intensidad inicial del salto;
- gravedad: rapidez con que el personaje vuelve al suelo;
- tiempoSalto: tiempo acumulado desde el inicio del salto.

10.4. Paso 12: iniciar el salto

```

1 private void controlarSalto()
2 {
3     if (Greenfoot.isKeyDown("space") && !saltando)
4     {
5         iniciarSalto();
6     }
7 }
```

Luego:

```

1 private void iniciarSalto()
2 {
3     saltando = true;
4     tiempoSalto = 0;
5     yInicial = getY();
6 }
```

```

7     estado = Estado.SALTANDO;
8 }

```

La condición:

```

1 !saltando

```

evita iniciar un segundo salto mientras el primero todavía está activo.

10.5. Paso 13: calcular la posición vertical

```

1 private void actualizarSalto()
2 {
3     if (saltando)
4     {
5         tiempoSalto++;
6
7         double y =
8             yInicial
9             - velocidadInicial * tiempoSalto
10            + 0.5 * gravedad
11            * tiempoSalto * tiempoSalto;
12
13        if (y >= yInicial && tiempoSalto > 1)
14        {
15            y = yInicial;
16
17            saltando = false;
18            estado = Estado.QUIETO;
19        }
20
21        setLocation(
22            getX(),
23            (int)y
24        );
25    }
26 }

```

Cada ciclo incrementa:

$$t = 0, 1, 2, 3, \dots$$

y la posición vertical vuelve a calcularse.

10.6. Paso 14: observar el salto matemáticamente

Si:

$$y_0 = 400, \quad v_0 = 12, \quad g = 1$$

entonces:

$$y(t) = 400 - 12t + 0.5t^2$$

Para $t = 0$:

$$y(0) = 400$$

Para $t = 5$:

$$y(5) = 400 - 60 + 12.5 = 352.5$$

El personaje sube porque:

$$352.5 < 400$$

Para $t = 12$:

$$y(12) = 400 - 144 + 72 = 328$$

Cerca de este instante se encuentra en su mayor altura.

Para $t = 20$:

$$y(20) = 400 - 240 + 200 = 360$$

El personaje ya está descendiendo.

Finalmente:

$$y(24) = 400 - 288 + 288 = 400$$

El personaje regresa al suelo.

11. Parte VI: integrar el comportamiento

11.1. Paso 15: definir el ciclo principal

El método `act()` puede organizarse como:

```

1 public void act()
2 {
3     controlarMovimientoHorizontal();
4     controlarSalto();
5     actualizarSalto();
6     actualizarAnimacion();
7 }
```

La estructura conceptual queda:

Entrada → Estado → Movimiento → Animación

12. Código completo de la clase Jugador

```
1 import greenfoot.*;
2
3 public class Jugador extends Actor
4 {
5     private enum Estado
6     {
7         QUIETO,
8         CAMINANDO_DERECHA,
9         CAMINANDO_IZQUIERDA,
10        SALTANDO
11    }
12
13    private Estado estado = Estado.QUIETO;
14
15    private GreenfootImage imagenQuieto;
16    private GreenfootImage[] caminar;
17
18    private int frameActual = 0;
19    private int contadorAnimacion = 0;
20
21    private boolean saltando = false;
22    private int tiempoSalto = 0;
23    private int yInicial;
24
25    private double velocidadInicial = 12.0;
26    private double gravedad = 1.0;
27
28    public Jugador()
29    {
30        cargarImagenes();
31    }
32
33    private void cargarImagenes()
34    {
35        imagenQuieto = new GreenfootImage("idle.png");
36
37        caminar = new GreenfootImage[4];
38
39        caminar[0] = new GreenfootImage("walk1.png");
40        caminar[1] = new GreenfootImage("walk2.png");
41        caminar[2] = new GreenfootImage("walk3.png");
42        caminar[3] = new GreenfootImage("walk4.png");
43
44        setImage(imagenQuieto);
45    }
46
47    public void act()
48    {
49        controlarMovimientoHorizontal();
50        controlarSalto();
51        actualizarSalto();
52        actualizarAnimacion();
53    }
54
55    private void controlarMovimientoHorizontal()
56    {
57        if (Greenfoot.isKeyDown("right"))
58        {
59            setLocation(getX() + 4, getY());
```

```
60         if (!saltando)
61         {
62             estado = Estado.CAMINANDO_DERECHA;
63         }
64     }
65     else if (Greenfoot.isKeyDown("left"))
66     {
67         setLocation(getX() - 4, getY());
68
69         if (!saltando)
70         {
71             estado = Estado.CAMINANDO_IZQUIERDA;
72         }
73     }
74     else
75     {
76         if (!saltando)
77         {
78             estado = Estado.QUIETO;
79         }
80     }
81 }
82
83
84 private void controlarSalto()
85 {
86     if (Greenfoot.isKeyDown("space") && !saltando)
87     {
88         iniciarSalto();
89     }
90 }
91
92 private void iniciarSalto()
93 {
94     saltando = true;
95     tiempoSalto = 0;
96     yInicial = getY();
97
98     estado = Estado.SALTANDO;
99 }
100
101 private void actualizarSalto()
102 {
103     if (saltando)
104     {
105         tiempoSalto++;
106
107         double y =
108             yInicial
109             - velocidadInicial * tiempoSalto
110             + 0.5 * gravedad
111             * tiempoSalto * tiempoSalto;
112
113         if (y >= yInicial && tiempoSalto > 1)
114         {
115             y = yInicial;
116
117             saltando = false;
118             estado = Estado.QUIETO;
119         }
120     }
121 }
```

```
120         setLocation(  
121             getX(),  
122             (int)y  
123         );  
124     }  
125 }  
126  
127  
128 private void actualizarAnimacion()  
129 {  
130     if (estado == Estado.QUIETO)  
131     {  
132         setImage(imagenQuieto);  
133     }  
134     else if (estado == Estado.CAMINANDO_DERECHA)  
135     {  
136         animarCaminata(false);  
137     }  
138     else if (estado == Estado.CAMINANDO_IZQUIERDA)  
139     {  
140         animarCaminata(true);  
141     }  
142 }  
143  
144 private void animarCaminata(boolean izquierda)  
145 {  
146     contadorAnimacion++;  
147  
148     if (contadorAnimacion >= 6)  
149     {  
150         frameActual++;  
151  
152         if (frameActual >= caminar.length)  
153         {  
154             frameActual = 0;  
155         }  
156  
157         GreenfootImage imagen =  
158             new GreenfootImage(caminar[frameActual]);  
159  
160         if (izquierda)  
161         {  
162             imagen.mirrorHorizontally();  
163         }  
164  
165         setImage(imagen);  
166  
167         contadorAnimacion = 0;  
168     }  
169 }  
170 }
```

13. Conceptos de programación utilizados

Concepto	Aplicación
Clase	Jugador
Objeto	Instancia del jugador agregada al mundo
Atributos	Estado, velocidad, gravedad, imágenes
Métodos	Caminar, saltar, animar
Arreglos	Secuencia de sprites
Condicionales	Detección de teclas y estados
enum	Representación explícita de estados
Booleanos	Saber si el personaje está saltando
Casting	Conversión de <code>double</code> a <code>int</code>
Ecuación cuadrática	Trayectoria vertical del salto
Abstracción	Separación de responsabilidades

14. Experimentos de laboratorio

14.1. Experimento 1: modificar la altura del salto

Cambiar:

```
1 velocidadInicial = 12.0;
```

por:

```
1 velocidadInicial = 16.0;
```

Responder:

1. ¿Aumenta o disminuye la altura máxima?
2. ¿Permanece más tiempo en el aire?
3. ¿Qué parte de la ecuación fue modificada?

14.2. Experimento 2: modificar la gravedad

Comparar:

```
1 gravedad = 1.0;
```

con:

```
1 gravedad = 2.0;
```

Analizar la diferencia.

14.3. Experimento 3: velocidad horizontal

Comparar:

```
1 getX() + 4
```


con:

```
1 getX() + 8
```

14.4. Experimento 4: velocidad de animación

Modificar:

```
1 contadorAnimacion >= 6
```

utilizando los valores:

3, 6, 10, 15

y comparar visualmente los resultados.

15. Desafíos de extensión

15.1. Desafío 1: sprite específico de salto

Agregar:

jump.png

y conseguir:

QUIETO $\rightarrow$ *idle.png*

CAMINANDO $\rightarrow$ *walk1, \dots, walk4*

SALTANDO $\rightarrow$ *jump.png*

15.2. Desafío 2: separar SALTANDO y CAYENDO

Agregar un nuevo estado:

```
1 CAYENDO
```

Para:

$$y(t) = y_0 - v_0 t + \frac{1}{2} g t^2$$

el instante de mayor altura ocurre aproximadamente en:

$$t_{\text{máx}} = \frac{v_0}{g}$$

Si:

$$v_0 = 12, \quad g = 1$$

entonces:

$$t_{\text{máx}} = 12$$

Podría utilizarse:

```

1  if (tiempoSalto < 12)
2  {
3      estado = Estado.SALTANDO;
4  }
5  else
6  {
7      estado = Estado.CAYENDO;
8  }
```

Luego asociar:

jump.png
fall.png

a cada estado.

15.3. Desafío 3: plataformas

Crear:

```

1  public class Plataforma extends Actor
2  {
3  }
```

y evolucionar desde:

salto → regresar al mismo y

hacia:

salto → trayectoria → colisión → aterrizaje

16. Actividad de cierre: relacionar arquitectura y código

Completar la siguiente tabla después de terminar el laboratorio.

Driver o concern	Decisión tomada	Elemento de código
Respuesta visual		
Mantenibilidad		
Movimiento natural		
Fluidez		
Extensibilidad		
Consistencia		

17. Conclusión

El desarrollo realizado puede entenderse mediante dos cadenas principales.

Para la animación:

Teclado $\rightarrow$ Estado $\rightarrow$ Movimiento $\rightarrow$ Sprite

Para el salto:

Tecla espacio $\rightarrow$ Estado SALTANDO $\rightarrow t \rightarrow y(t) \rightarrow$ posición vertical

La actividad muestra que incluso un videojuego pequeño implica decisiones arquitectónicas. La calidad del resultado no depende solamente de que el personaje se mueva, sino de que exista coherencia entre control, comportamiento, representación visual, estructura del código y posibilidad de evolución futura.